LEARNHUB

Rapport de Projet Final

Plateforme d'apprentissage en ligne — MongoDB

Matiere : Base de donnees NoSQL-- MongoDB
Réalise par : Sanaa Goulahyane
Date : 4 mars 2026

Objectif :

Ce rapport documente la réalisation du projet LearnHub, une plateforme d'apprentissage en ligne.

Introduction :

Le projet LearnHub vise à modéliser une base de données MongoDB pour une plateforme d'apprentissage, insérer des données réalistes, implémenter des requêtes complexes et exposer une API REST. L'accent est mis sur MongoDB avec un emballage API.

Phase 1 — Modélisation & Seed Data

La base de données learnhub repose sur 5 collections MongoDB distinctes, chacune représentant une entité clé de la plateforme.

1.1 — Collections créées

Collection	Documents	Description
users	20	Étudiants et instructeurs, profil imbriqué, tableau de skills
courses	15	Cours publiés ou non, métadonnées, rating, catégorie, difficulté
lessons	45	Leçons liées à un cours , ordre, ressources
enrollments	25	Inscriptions avec statut, progression et informations de paiement
reviews	20	Avis utilisateurs sur les cours, note 1-5, isVerified, helpfulCount

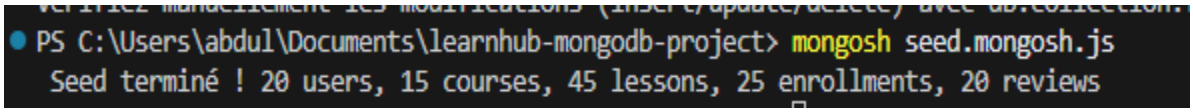
1.2 — Choix de modélisation

- Documents imbriqués pour profile (users) et metadata/rating (courses) : données fortement couplées, toujours lues ensemble
- Références par ObjectId entre collections : enrollments/reviews référencent userId et courseId
- Tableaux natifs MongoDB : skills (users), tags (courses), completedLessons (progress), resources (lessons)
- Champs dénormalisés : enrollmentCount dans courses, totalCoursesEnrolled dans users — mis à jour via $inc pour éviter des agrégations coûteuses

1.3 — Données de seed

Le script seed.mongosh.js insère 110 documents au total avec des données réalistes et cohérentes. Les instructeurs sont référencés par ObjectId dans les cours, et les ObjectId des leçons sont utilisés dans les tableaux completedLessons des inscriptions.

Capture d'écran :



Phase 2 — Requêtes MongoDB (queries.mongosh.js)

Le fichier queries.mongosh.js contient 30 requêtes commentées, ajoutant des print() pour les outputs visibles, organisées en 3 rubriques. Tous les opérateurs obligatoires sont utilisés au moins une fois.

2.1 — Opérations CRUD (10 requêtes)

#	Opération	Opérateur	Description
1	**INSERT**	insertOne	Insérer un nouvel étudiant avec tous ses champs
2	**INSERT**	insertMany	Insérer 3 cours en une seule opération
3	**UPDATE**	$set + notation pointée	Modifier la ville du profil d'un utilisateur
4	**UPDATE**	$inc	Incrémenter enrollmentCount d'un cours de 1
5	**UPDATE**	$push	Ajouter un skill au tableau d'un utilisateur
6	**UPDATE**	$pull	Retirer un tag du tableau d'un cours
7	**UPDATE**	updateMany	Désactiver utilisateurs inactifs depuis 6 mois
8	**UPSERT**	upsert: true	Créer ou mettre à jour le profil d'un utilisateur
9	**DELETE**	deleteOne	Supprimer une review spécifique par _id
10	**DELETE**	deleteMany	Supprimer toutes les inscriptions cancelled

2.2 — Requêtes de sélection (15 requêtes)

#	Opérateur(s)	Description
11	**$gte, $lte**	Cours dont le prix est entre 20€ et 80€
12	**$in**	Cours dans les catégories Database ou Web
13	**$ne**	Cours dont la difficulté n'est pas advanced
14	**$and**	Utilisateurs actifs ET de rôle student

15	**$or**	Cours gratuits OU avec note >= 4.5
16	**$exists, $ne null**	Reviews dont updatedAt existe et n'est pas null
17	**notation pointée**	Utilisateurs habitant à Paris (profile.city)
18	**$and explicite**	Cours publiés avec note moyenne >= 4
19	**$nin**	Inscriptions ni cancelled ni paused
20	**projection inclusion**	Titre et prix des cours seulement (sans _id)
21	**projection exclusion**	Tous les champs users sauf le profil
22	**sort + limit**	Top 5 cours les mieux notés
23	**sort ASC**	Tous les cours triés par prix croissant
24	**skip + limit**	Page 2 du catalogue (10 résultats par page)
25	**countDocuments**	Compter le nombre de cours publiés

2.3 — Requêtes Métier (5 requêtes)

26	**Inscription complète**	Vérifier user/cours, vérifier doublon, insertOne, $inc ×2
27	**Catalogue filtré**	$and multi-critères, projection, sort popularité, pagination
28	**Progression**	$push leçon + $set recalcul du pourcentage
29	**Suppression cascade**	deleteOne cours, deleteMany leçons/reviews, updateMany inscriptions
30	**Dashboard**	findOne user + find inscriptions actives + find derniers avis

Capture d'écran :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

  category: 'Mobile',
  difficulty: 'intermediate',
  price: 94.7,
  tags: [
    'mongodb',
    'nosql',
    'fullstack'
  ],
  metadata: {
    duration: 1357,
    totalLessons: 19,
    language: 'fr'
  },
  rating: {
    average: 4.9,
    count: 75
  },
  isPublished: true,
  enrollmentCount: 117,
  createdAt: ISODate('2024-09-30T23:00:00.000Z'),
  updatedAt: ISODate('2026-03-02T12:44:30.166Z')
}

17. Utilisateurs habitant à Paris
{
  _id: ObjectId('69a5862e16489ce0ed7c2907'),
  firstName: 'Alice',
  lastName: 'Martin',
  email: 'alice.martin@email.com',
  role: 'instructor',
  profile: {
```

Phase 3 — API REST Node.js / Express

L'API REST est construite avec Node.js et Express. Elle expose les requêtes MongoDB de la Phase 2 via des endpoints HTTP organisés par collection. La connexion à MongoDB est centralisée dans un module db.js réutilisé dans toutes les routes.

3.1 — Architecture

- server.js — Point d'entrée Express, montage des routes
- db.js — Connexion MongoDB singleton
- routes/users.js — CRUD utilisateurs + dashboard
- routes/courses.js — CRUD cours + suppression cascade
- routes/enrollments.js — Inscriptions + progression
- routes/reviews.js — Avis + recalcul de moyenne
- routes/lessons.js — CRUD leçons

3.2 — Endpoints

Méthode	Route	Opération	Description
POST	/api/users	insertOne	Créer un utilisateur
GET	/api/users	find + filtres	Lister (role, city, active)
GET	/api/users/:id	findOne	Récupérer un utilisateur
PATCH	/api/users/:id	$set/$push/$pull/$inc	Modifier partiellement
PUT	/api/users/:id/profile	upsert: true	Créer/mettre à jour profil
GET	/api/users/:id/dashboard	findOne + find ×2	Dashboard complet
POST	/api/courses/bulk	insertMany	Insérer plusieurs cours
GET	/api/courses	find + sort + skip	Catalogue filtré paginé
GET	/api/courses/:id	findOne	Détail d'un cours
DELETE	/api/courses/:id	deleteOne + cascade	Suppression cascade
POST	/api/enrollments	insertOne + $inc ×2	Inscription complète
PATCH	/api/enrollments/:id/progress	$push + $set	Marquer leçon complétée
POST	/api/reviews	insertOne + $set	Avis + recalcul moyenne
DELETE	/api/reviews/:id	deleteOne	Supprimer un avis
GET	/api/lessons	find + sort	Leçons d'un cours

Phase 4 — Logique Métier

Cinq endpoints combinent plusieurs opérations MongoDB séquentielles pour implémenter des fonctionnalités métier concrètes.

POST /api/enrollments — Inscrire un utilisateur	
1.	findOne — vérifier que l'utilisateur existe
2.	findOne — vérifier que le cours existe et est isPublished: true
3.	findOne — vérifier qu'il n'est pas déjà inscrit (doublon)
4.	insertOne — créer l'inscription avec statut active et progression 0%
5.	updateOne + $inc — incrémenter enrollmentCount du cours
6.	updateOne + $inc — incrémenter totalCoursesEnrolled de l'utilisateur

POST /api/reviews — Laisser un avis	
1.	findOne — vérifier que l'inscription au cours existe
2.	findOne — vérifier l'absence de doublon d'avis
3.	insertOne — créer la review (isVerified si status = completed)
4.	find + updateOne + $set — recalculer la moyenne du cours

PATCH /api/enrollments/:id/progress — Marquer une leçon complétée	
1.	findOne — vérifier que l'inscription existe
2.	updateOne + $push — ajouter le lessonId aux leçons complétées
3.	updateOne + $set — recalculer progress.percentage
4.	Si 100% — $set status: completed + completedAt: new Date()

DELETE /api/courses/:id — Suppression cascade	
1.	deleteOne — supprimer le cours
2.	deleteMany — supprimer toutes les leçons du cours
3.	deleteMany — supprimer toutes les reviews du cours
4.	updateMany + $set — annuler les inscriptions liées (status: cancelled)

GET /api/users/:id/dashboard — Dashboard utilisateur

1.	findOne — récupérer l'utilisateur avec projection (sans avatar)
2.	find — ses inscriptions actives (filtre userId + status: active)
3.	find + sort + limit — ses 5 derniers avis (tri par createdAt DESC)
4.	Assembler la réponse : { user, activeEnrollments, recentReviews }

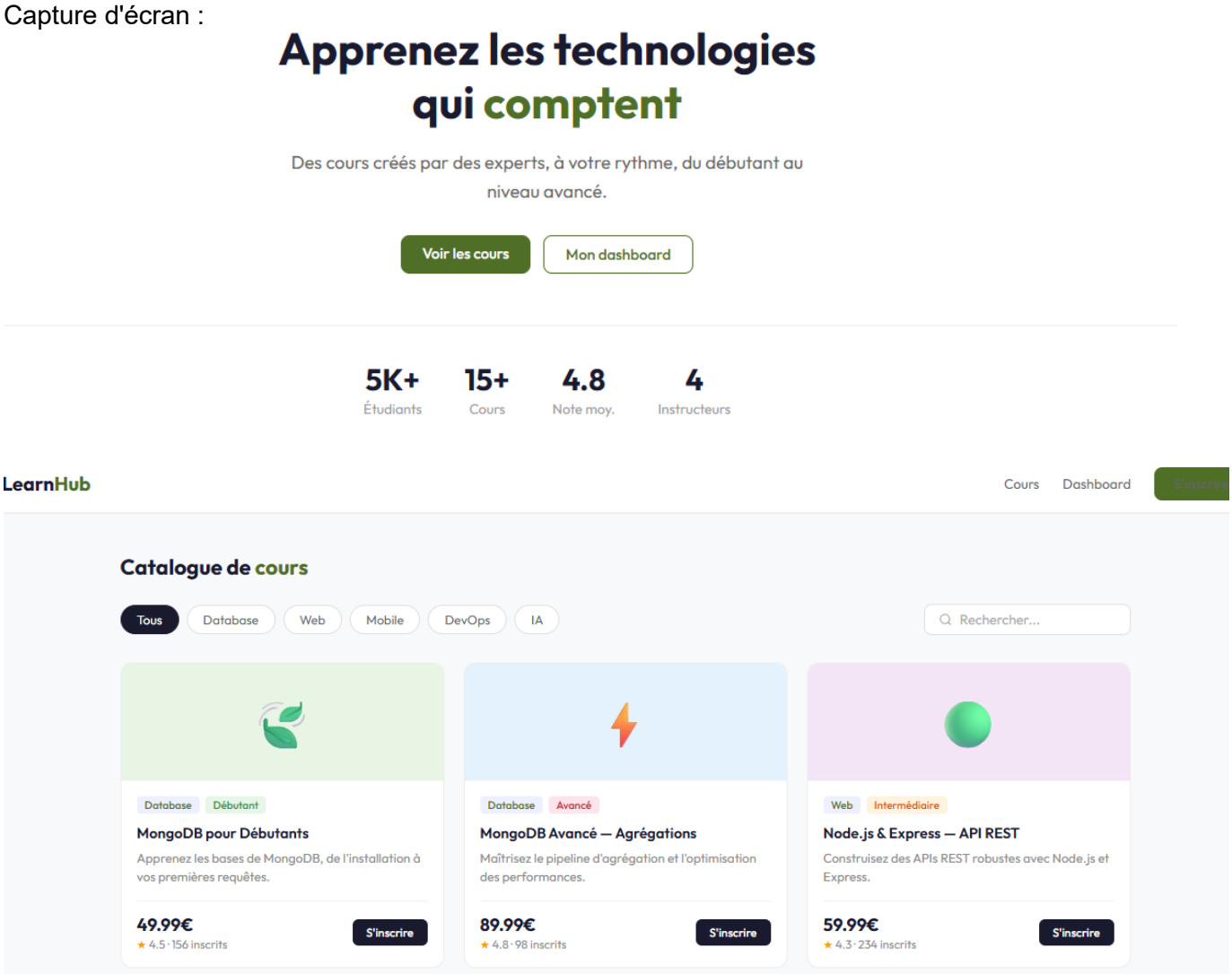
Phase 5 — Bonus : Frontend

Un frontend simple et fonctionnel a été réalisé en HTML, CSS et JavaScript .

Fonctionnalités implémentées

- Page d'accueil avec hero section et statistiques
- Catalogue de cours avec filtres par catégorie (Database, Web, Mobile, DevOps, AI)
- Barre de recherche en temps réel sur les titres et descriptions
- Modal d'inscription avec validation des champs
- Modal dashboard avec barres de progression animées
- Toast de confirmation après inscription
- Design responsive (mobile, tablette, desktop)

Capture d'écran :



Conclusion

Le projet LearnHub a été conduit en quatre phases majeures auxquelles s'ajoute un bonus frontend. La base de données MongoDB a été modélisée de façon réaliste avec des choix justifiés . Les 30 requêtes de la Phase 2 couvrent l'intégralité des opérateurs exigés et sont commentées. L'API REST Node.js/Express expose ces requêtes via 15 endpoints. Enfin, les 5 endpoints métier de la Phase 4 démontrent la capacité à enchaîner plusieurs opérations MongoDB en logique applicative cohérente.